Documento de Decisiones Técnicas · Hogar Andino

1. Resumen

Implementamos dos dominios del diseño de la Evaluación 3, que hicimos con ArchiMate y DDD. C1, Fuentes y metadatos técnicos, usa arquitectura hexagonal y está escrito en C# con .NET 10. C2, Atributos de negocio, usa microservicios y está escrito en Python con FastAPI. Los demás contextos del diseño se simulan para mostrar cómo se comunican con estos dos.

Este documento explica por qué tomamos cada decisión de arquitectura, base de datos, patrones y despliegue. El equipo está formado por María Alexandra Jiménez Suárez, Juan José Álvarez Restrepo, Carolina Ramírez Lotero y Natalia Giraldo Morales.

Dato	**C1**	**C2**
Arquitectura	Hexagonal	Microservicios
Lenguaje	C# / .NET 10	Python 3.13 / FastAPI
Pruebas automáticas que pasan	71 de 71	51 de 51
Fidelidad al diseño (sección 10)	93,5 %	94,4 %

2. Por qué C1 es hexagonal

En el diseño, C1 es el contexto que más se relaciona con el mundo exterior. Lee las estructuras de motores relacionales (A1) y del CRM a través de su proveedor (A2), recibe avisos de cambio de los proveedores (A3), le pregunta a C7 si un usuario tiene permiso y avisa con eventos a C2 y C6. Casi todo lo que hace consiste en traducir lo que dicen sistemas ajenos al lenguaje del dominio.

La arquitectura hexagonal responde justo a esa situación. Deja las reglas del negocio en el centro y pone todo lo externo detrás de puertos, que son interfaces, con adaptadores que las implementan. El diseño ya pedía una capa anticorrupción para que los sistemas externos no contaminen el modelo, y en la hexagonal esa capa son los adaptadores con sus traductores. Con ella podemos cambiar un lector simulado por uno real, o SQLite por otra base, tocando solo el host, y probar las reglas del dominio sin servidor ni base de datos.

Descartamos los microservicios para C1 porque no necesita escalar por separado ni tiene consumidores que exijan un contrato propio. Convertirlo en un servicio independiente solo sumaría complejidad de red sin darle nada a cambio.

3. Por qué C2 es microservicios

C2 es el dominio principal del diseño y cuatro contextos dependen de él, que son C3, C4, C5 y C6. Tiene su propio modelo y su propia base de datos, y publica eventos que otros contextos consumen. Es un dominio con vida propia, y eso es lo que describe un microservicio.

Hay tres razones que lo sostienen. La primera es la autonomía, porque C2 tiene su ciclo de vida y no comparte tablas con nadie. La segunda es la comunicación por eventos, ya que al registrar un atributo o actualizar un metadato C2 publica un evento que otros esperan. La tercera es el volumen de lectura, porque varios contextos consultan los atributos con frecuencia, lo que justifica una caché y la posibilidad de escalar este servicio sin tocar a C1. Además, lo que sale de C2 es un contrato público versionado y no sus clases internas, así que podemos cambiarlo sin afectar a C1.

También consideramos usar hexagonal en C2. Serviría para organizar el dominio, pero no nos obligaría a resolver los problemas propios de un microservicio, como la mensajería confiable, la idempotencia y los contratos versionados.

4. Criterios que pesaron en la decisión

Criterio	**C1**	**C2**	**Qué favorece**
Sistemas externos con los que habla (A1, A2, A3, C7)	Muchos	Pocos (solo el inventario manual, A4)	Hexagonal para C1, porque los puertos aíslan cada sistema
Formatos externos que debe traducir	Altos, con estructuras técnicas de motores y CRM	Bajos	Hexagonal para C1, con traductores anticorrupción
Contextos que lo consumen	Pocos (C2 y C6)	Varios (C3, C4, C5 y C6)	Microservicio para C2, con contratos versionados
Volumen de lectura	Bajo, con importaciones puntuales	Alto, con consultas frecuentes	Microservicio para C2, con caché y escalado propios
Necesidad de modelo y datos propios	Media	Alta	Microservicio para C2, con una base por servicio
Garantía de entrega de eventos	Media	Alta, porque otros dependen de sus eventos	Microservicio para C2, con outbox e inbox

La tabla muestra que los dos dominios tienen necesidades opuestas. C1 está rodeado de sistemas externos con formatos propios y casi nadie depende de él, así que lo que necesita es aislarse, y eso lo da la hexagonal. C2 casi no habla con sistemas externos, pero muchos contextos lo consultan y dependen de sus eventos, así que necesita contratos estables, datos propios y entrega confiable, y eso lo dan los microservicios. Si invirtiéramos la asignación, C1 pagaría la complejidad de red sin tener quién la justifique y C2 quedaría sin la mensajería y los contratos que su papel exige.

5. Lenguajes

Dominio	Lenguaje	Por qué
C1	C#	Por preferencia del equipo y porque lo entendemos mejor. Además, su tipado fuerte y sus interfaces explícitas hacen que los puertos se vean claros.
C2	Python (FastAPI)	Permite construir rápido un servicio HTTP pequeño, y FastAPI valida las entradas, algo que encaja con un microservicio liviano.

6. Base de datos: SQLite en los dos dominios

Elegimos una base de datos SQL, con SQLite en ambos dominios y un archivo independiente para cada servicio. Los datos son estructurados y se relacionan entre sí, como la fuente con sus autorizaciones, la versión con sus columnas o el atributo con sus versiones. Además necesitamos transacciones. En C1, importar una estructura guarda la versión y actualiza la fuente, y las dos cosas deben guardarse juntas o no guardarse. En C2, el atributo y su mensaje del outbox se guardan en la misma transacción. Una base NoSQL se elige sobre todo por esquemas flexibles o escrituras masivas, y ninguna de las dos cosas nos hace falta.

Entre las bases SQL escogimos SQLite sobre PostgreSQL porque no hay que instalar nada, cada servicio tiene su propio archivo, lo que refleja el principio de una base por microservicio, y cualquier integrante del equipo puede ejecutar todo en su máquina. Como el acceso está detrás de puertos (repositorio y unidad de trabajo), cambiar de base implicaría tocar solo los adaptadores.

Cada servicio es dueño de sus tablas y nadie más las lee. Lo que C2 necesita de C1 le llega por evento.

Servicio	**Tablas**	**Qué guardan**
C1	fuentes, autorizaciones, credenciales	La fuente, quién autorizó leer su estructura y las credenciales cifradas
C1	versiones, columnas_version	Cada lectura de la estructura (tablas y columnas), que no se modifica una vez creada
C1	cambios, diferencias, revisiones_cambio	Los cambios detectados entre dos versiones, sus diferencias y su revisión
C1	avisos	Avisos de cambio que llegan de los proveedores (A3)
C2	atributos	El estado actual de cada atributo de negocio
C2	atributo_versiones	El historial, donde cada escritura guarda el memento serializado como una versión
C2	outbox, inbox	Eventos por publicar y eventos ya recibidos, para no procesarlos dos veces
C2	fuentes_locales	Copia local de las fuentes que C1 anunció, para validar que un atributo apunte a una fuente existente

Cada comando es una transacción. En C1 la unidad de trabajo confirma las escrituras juntas o las deshace. En C2 el paso de transacción del pipeline hace lo mismo y garantiza que el atributo y su evento en el outbox se guarden a la vez. Las versiones nunca se modifican, solo se agregan, porque el gobierno de datos necesita trazabilidad para saber qué se declaró y cuándo, y porque así la caché de versiones de C1 no necesita invalidarse. Por último, C2 detecta ediciones simultáneas con concurrencia

optimista, de modo que la segunda operación sobre el mismo atributo recibe un error en lugar de pisar el cambio de la primera.

7. Patrones de diseño

Cada patrón resuelve un problema concreto del diseño. En C1 giran alrededor de aislar el núcleo de lo externo, y en C2 alrededor de que los eventos y los datos sean confiables.

En C1, que es hexagonal, usamos estos patrones.

Patrón	**Dónde**	**Para qué**
Puertos y adaptadores	Puertos/ y proyectos C1.Adaptadores.*	Aísla el núcleo de HTTP, SQLite y sistemas externos
Mediador, comando y pipeline	Mediador, IPasoDeComando	Un solo punto de entrada, con los pasos transversales como los permisos en piezas aparte
Repositorio	Uno por tipo de dato	El dominio habla de objetos y no de SQL
Unidad de trabajo	UnidadDeTrabajoSqlite	Confirma varias escrituras juntas o hace rollback
Traductores anticorrupción	Traductores, lectores A1 y A2	Convierten lo externo al lenguaje del dominio
Decorator	Caché de versiones	Agrega caché sin modificar el repositorio
Memento	MementoDeFuente	Deshace en memoria si guardar falla
Raíz de composición	C1.Host/Program.cs	Único lugar que conoce las clases concretas
Lenguaje publicado (Published Language)	C1.Contracts	Contratos V1 versionados hacia C2 y C6

En C2, que es un microservicio, usamos estos otros.

Patrón	**Dónde**	**Para qué**
CQRS, mediador y pipeline	mediador.py, comportamientos.py	Separa escribir de leer, con los pasos Registro, Validación, Despacho y Transacción

Outbox	Tabla outbox y DespachadorDeOutbox	No se pierde un evento ni se publica uno que no se guardó
Inbox y consumidor idempotente	Tabla inbox	Si un evento llega dos veces, se procesa una sola
Contratos versionados	contracts/v1.py	El exterior ve contratos públicos y no el dominio interno
Template Method	ManejadorDeModificacion	Las modificaciones comparten el mismo esqueleto
Specification	especificaciones.py	La regla de atributo incompleto, combinable y reutilizable
Decorator	LecturaAtributosConCache	Caché sin tocar la lectura real
Memento	AtributoMemento	Deshace cambios y guarda el historial de versiones
Concurrencia optimista	Repositorio	Detecta si dos operaciones cambian el mismo atributo
Errores estándar (ProblemDetails)	api/problemas.py	Respuestas de error uniformes

Los puertos y adaptadores de C1 existen porque los sistemas externos cambian y no deben contaminar el dominio. Sin ellos, los manejadores llamarían directamente a cada sistema y las pruebas necesitarían sistemas reales. El costo es tener más interfaces y archivos.

El mediador con pipeline, presente en los dos dominios, permite que los permisos, la validación y la transacción se apliquen a todos los casos de uso sin repetirse. Su costo es un salto más para seguir el flujo y, en C1, que el mediador use un service locator para encontrar los manejadores. La unidad de trabajo de C1 y el paso de transacción de C2 aseguran que un comando que escribe en varias tablas quede todo o nada, a cambio de que la interfaz de la unidad de trabajo crezca con cada repositorio nuevo.

El outbox de C2 evita que una caída entre guardar el dato y publicar el evento pierda el evento, y el inbox evita procesar dos veces un evento repetido. Cuestan una tabla extra cada uno y un despachador. C1 publica sus eventos después de confirmar, sin outbox, y aceptamos ese riesgo porque nadie depende allí de una entrega garantizada.

Los traductores anticorrupción evitan que los nombres y formatos de cada proveedor entren al modelo, a cambio de una clase por traducción. El Decorator permite agregar la caché sin tocar el repositorio ni la lectura real, a cambio de una clase envoltorio. El Memento permite volver al estado anterior si guardar falla y, en C2, registrar el historial de versiones, a cambio de serializar el estado de cada objeto.

Por último, en C2 la especificación del atributo incompleto evita condiciones sueltas por los manejadores, el Template Method evita tener tres manejadores casi iguales, y CQRS mantiene separadas las lecturas frecuentes, que llevan caché, de las escrituras, que llevan transacción.

8. Caché, Memento, cifrado y despliegue

Caché

En C2 usamos una caché con vencimiento para consultar atributos, porque varios contextos llaman muchas veces a esa consulta y el dato cambia poco. La lectura con caché envuelve a la lectura real, como un Decorator. Al consultar un atributo busca primero en la caché y, si no está, lo lee de la base y lo guarda por un tiempo configurable que por defecto es de 60 segundos. Si el atributo no existe no guarda nada, y los listados y las versiones se leen directo de la base. La caché se invalida cuando el bus entrega un evento de C2, como el registro de un atributo o la actualización de su metadato, y cuenta aciertos y fallos para poder comprobar que funciona. Con varias réplicas habría que cambiarla por Redis, cambiando solo el adaptador.

En C1 la caché envuelve al repositorio de versiones de estructura. Como una versión nunca cambia una vez creada, esta caché no necesita invalidarse.

Estado con Memento

En C2, el atributo genera un memento antes de modificarse y, si algo falla, vuelve a su estado anterior. Además, cada escritura guarda su memento serializado como una versión, y así cada cambio crea una versión. En C1, la fuente genera su memento antes de importar y, si guardar falla, se restaura el estado anterior y se descartan los eventos que ya había levantado.

Cifrado simple

En C1 se cifran con AES-GCM las credenciales de las cuentas de lectura de las fuentes, que se guardan cifradas y el dominio nunca ve. La llave de 256 bits sale de un secreto con PBKDF2 (SHA-256, 100 000 iteraciones), y cada cifrado usa un valor aleatorio distinto. En C2 se cifran con Fernet la definición y la política de calidad de cada atributo, porque el propio diccionario puede revelar dónde hay información

sensible. Fernet combina AES de 128 bits en modo CBC con una firma HMAC, y su llave sale de la frase secreta con PBKDF2 (SHA-256, 200 000 iteraciones).

En los dos casos el secreto viene de una variable de entorno y no del código. Cada servicio usa la suya, C1_CLAVE_CIFRADO y C2_CLAVE_CIFRADO, así que conocer una clave no sirve para descifrar los datos del otro servicio. Solo en modo de desarrollo existe una clave de prueba, y fuera de él el servicio no arranca sin la variable.

El cifrado protege contra quien consiga el archivo SQLite o un respaldo sin la clave. Como AES-GCM y Fernet autentican el dato, si alguien lo altera, o si se usa una clave equivocada, el descifrado falla en lugar de devolver basura. No protege contra quien tenga acceso al servidor en ejecución y a sus variables de entorno, y el resto de las columnas va en claro porque hay que buscarlas y compararlas. La sal de PBKDF2 es fija en los dos servicios por simplicidad académica, y en producción el secreto vendría de un gestor de secretos.

Despliegue con Docker

C1 se empaqueta en una imagen de dos etapas, una que compila con el SDK de .NET y otra que ejecuta con la imagen liviana de ASP.NET. C2 usa una imagen de Python 3.13 que instala primero las dependencias y luego copia el código. Los dos corren con un usuario sin privilegios, guardan su base en un volumen en /data y tienen un healthcheck.

Un docker-compose.yml levanta ambos servicios. C2 arranca primero y debe estar sano antes de que arranque C1, que le envía sus eventos por HTTP. Las claves de cifrado llegan desde un archivo .env y no están dentro de las imágenes.

9. Cómo simulamos los otros contextos

El diseño tiene siete contextos y solo implementamos C1 y C2. Los demás se simulan detrás de los mismos puertos que usaría el contexto real, de modo que pasar a uno real consista en reemplazar un adaptador sin tocar el dominio. Lo único que viaja de verdad entre dos dominios es el evento Fuente registrada, que C1 envía a C2 por HTTP cuando se define la variable C2_URL.

C7, el control de acceso, se simula con VerificadorDePermisosSimuladoC7, una matriz de permisos por rol. Los roles admin y ti pueden ejecutar las cinco operaciones que C1 protege, consulta y oficial no pueden ejecutar ninguna, y cada negación queda registrada. Usamos una matriz y no un simulador que siempre diga que sí, porque este último escondería los errores de permisos. La identidad del usuario llega en el encabezado X-Usuario, ya que la autenticación no forma parte del diseño de C1.

C6, el diccionario y reporte, se simula con un destino en memoria que recibe y anota los eventos de C1 y de C2. En C2, C4 recibe el evento de metadato de negocio actualizado

y C6 recibe el de atributo registrado y el de metadato actualizado. C4, el de responsables, se simula con ResponsabilidadSimuladaC4, que responde si un atributo tiene responsable, y con eso C2 aplica la regla de que un atributo con datos personales exige responsable. Los sistemas externos A1 y A2 se simulan con los lectores LectorA1MotorRelacional y LectorA2Crm, que devuelven estructuras de ejemplo. A3 y A4 son entradas HTTP reales, con avisos de cambio de prueba y con el inventario manual.

C3 y C5 no tienen simulador. En el diseño solo consultan el servicio abierto de C2 y, como no los implementamos, C2 ofrece esas consultas y sus contratos pero ningún componente las usa.

Si C2 no responde cuando C1 le envía un evento, el comando de C1 termina bien, porque el cambio ya estaba guardado, y la entrega fallida queda anotada para poder revisarla. No se reintenta, porque C1 no usa outbox. En C2, el bus es un Observer en memoria y si un suscriptor falla, eso no afecta a quien publicó. Para ver la simulación, C1 tiene rutas en /v1/simulacion con la estructura simulada, los mensajes publicados, las entregas fallidas, lo que recibieron C6 y C2 y las denegaciones de C7, y en C2 la simulación se activa con la variable C2_SIMULACION, activa salvo en producción.

Origen y destino	**Qué viaja**	**Real o simulado**
C1 a C2	Evento Fuente registrada (contrato V1), por HTTP	Real si se define C2_URL, y si no, C2 se simula en memoria
C1 a C6	Eventos de C1	Simulado, C6 los anota
C7 a C1	Consulta síncrona de permiso	Simulado con matriz de permisos
C2 y C4	Consulta de si el atributo tiene responsable	Simulado
C2 a C4 y C6	Eventos de C2	Suscriptores simulados
A1 y A2 a C1	Estructura de motores y CRM	Lectores simulados
A3 a C1	Aviso de cambio del proveedor, por POST a /v1/avisos-de-cambio	Entrada real con avisos de prueba
A4 a C2	Inventario manual, por POST a /v1/integracion/inventario	Entrada real

El código también conserva las reglas del diseño. Sin autorización no se importa una fuente, y solo se registra un cambio si hay diferencia entre versiones. La expresión regular solo se valida compilándola y nunca se ejecuta contra datos. Un atributo solo existe si su fuente existe en C1 y no se repite en la misma fuente. Lo que falta se muestra como faltante, de modo que sin definición el atributo queda incompleto, y cada cambio crea una versión.

10. Fidelidad al diseño

Validamos la fidelidad comparando el diseño de la Evaluación 3 con lo que quedó en el código. Al analizar lo que faltó, medimos los porcentajes teniendo en cuenta cuánto se planeó y cuánto se logró implementar, y por eso son una aproximación propia y no una medida oficial. El resultado es un 93,5 % en C1 y un 94,4 % en C2, es decir un 93,9 % entre los dos, por encima del 90 % que se exige.

Lo que falta para llegar al 100 % corresponde a las funciones que dependen de otros contextos, como solicitar acceso, registrar impacto, vincular columna y notificar al responsable, y a unos pocos ajustes frente al diseño. C1 publica sus eventos después de confirmar y sin outbox, C2 conserva la regla de atributo incompleto y la identidad del usuario llega por un encabezado en lugar de una autenticación completa.